

EECS E4750: Heterogeneous Computing

Assignment 1 Report

Assignment 1 Primer

Fanghao Hu
UNI: fh2551

October 3, 2026

1 Experimental Setup

All experiments were run on a Google Cloud VM. The GPU environment used for the measurements is shown in Figure 1.

The screenshot shows the Google Cloud VM configuration interface. At the top, there are two dropdown menus: 'GPU type' set to 'NVIDIA L4' and 'Number of GPUs' set to '1'. Below these is a checkbox labeled 'Enable Virtual Workstation (NVIDIA GRID)' which is currently unchecked. A table lists available machine types with columns for Series, Description, vCPUs, Memory, and Cost. The 'G2' series is selected. Below the table, there is a 'Machine type' section with a description and a link to 'Learn more'. Two buttons, 'Preset' and 'Custom', are visible. A dropdown menu for 'Instance sizes' is open, showing a list of options: 'g2-standard-4', 'g2-standard-8', 'g2-standard-12', 'g2-standard-16', and 'g2-standard-32', each with its corresponding vCPU, memory, and GPU specifications.

Series	Description	vCPUs	Memory	Cost
G2	ML inference, graphics-intensive, and HPC workloads	4 - 32	16 - 128 GB	Int

Machine type
Choose a machine type with preset amounts of vCPUs and memory that suit most workloads. Or, you can create a custom machine for your workload's particular needs. [Learn more](#)

Preset Custom

Filter Instance sizes

- g2-standard-4
4 vCPU (2 core), 16 GB memory, 1 L4 GPU
- g2-standard-8
8 vCPU (4 core), 32 GB memory, 1 L4 GPU
- g2-standard-12
12 vCPU (6 core), 48 GB memory, 1 L4 GPU
- g2-standard-16
16 vCPU (8 core), 64 GB memory, 1 L4 GPU
- g2-standard-32
32 vCPU (16 core), 128 GB memory, 1 L4 GPU

Figure 1: GPU hardware on the Google Cloud VM.

2 Correctness Results

The correctness result of PyOpenCL and PyCUDA are shown in Figures 2 and 3.

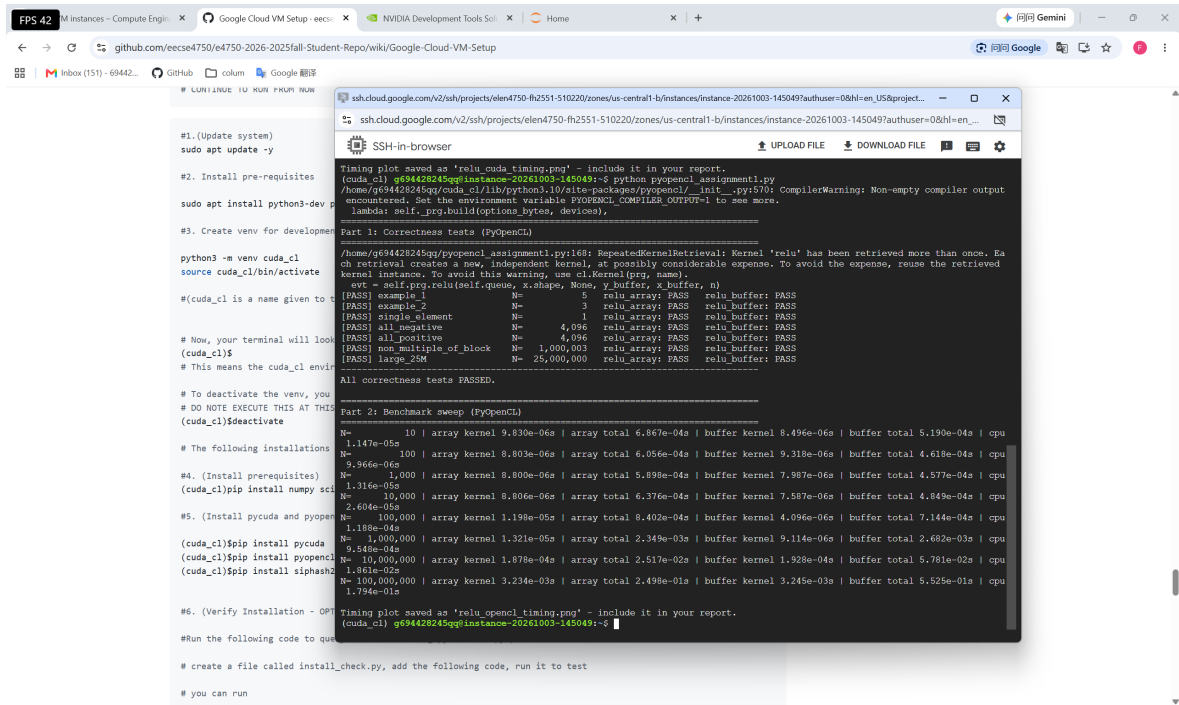


Figure 2: PyOpenCL correctness-test output.

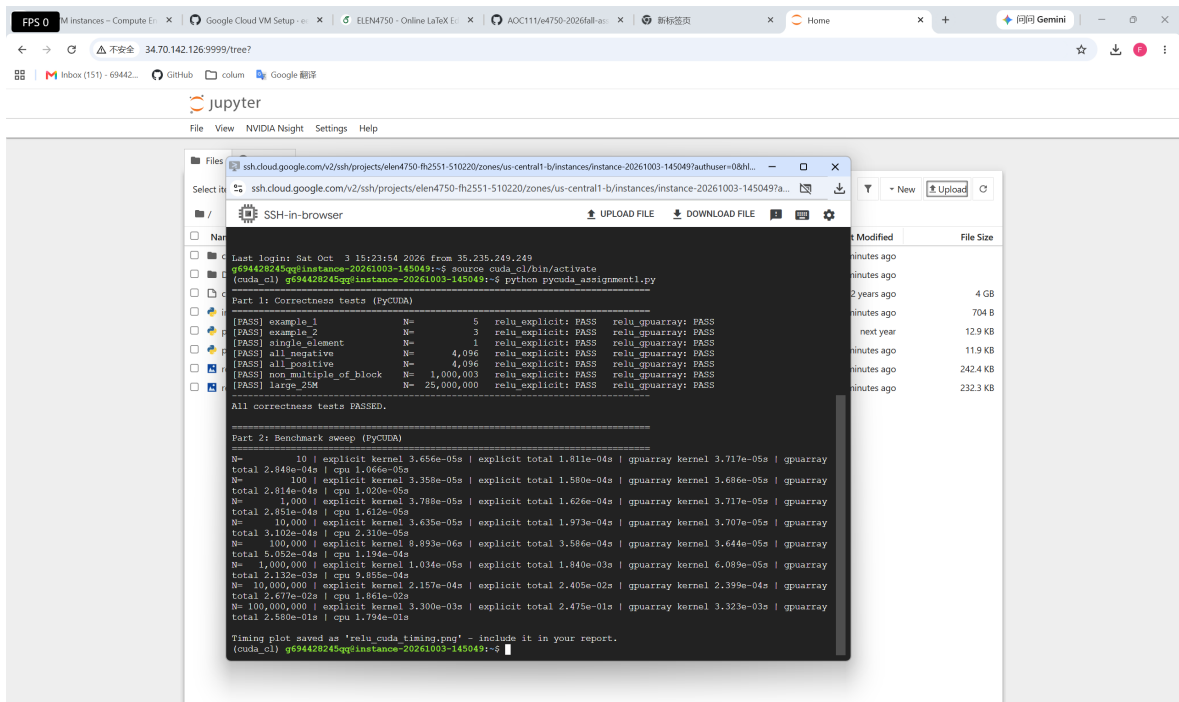


Figure 3: PyCUDA correctness-test output.

3 Timing Results and Analysis

3.1 PyOpenCL

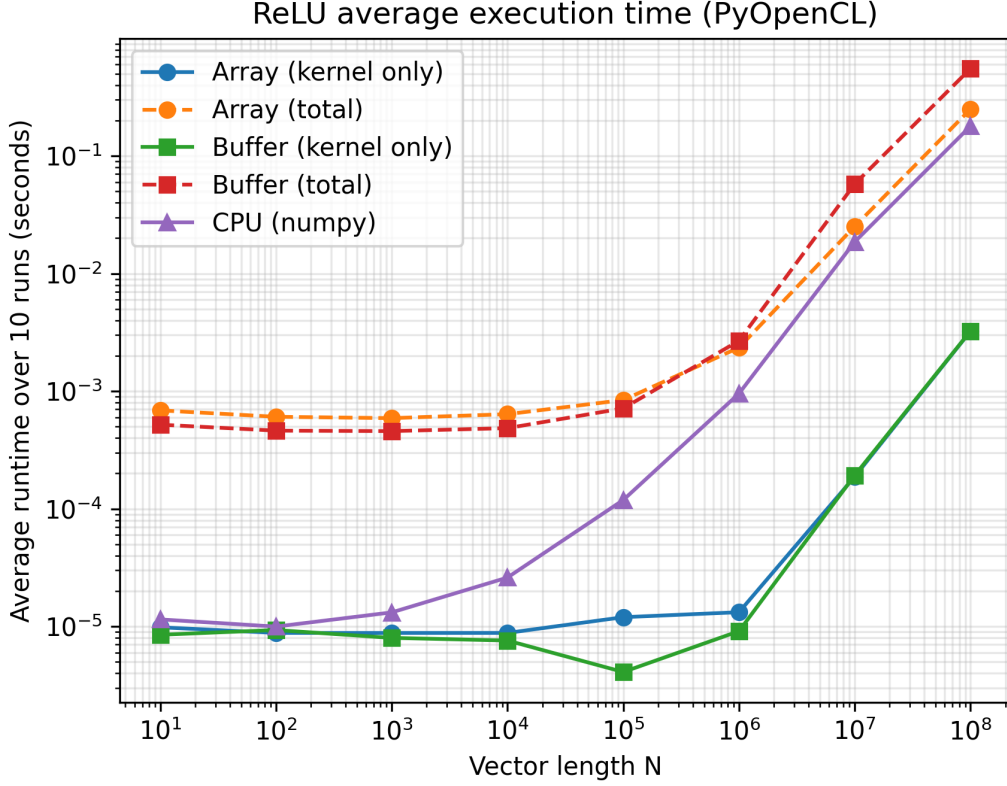


Figure 4: Average PyOpenCL ReLU execution time over 10 runs.

Figure 4 shows that kernel time stays near 10^{-5} seconds as N increases from 10^1 to 10^6 and then increases with vector size. The end-to-end curves remain near 10^{-3} seconds for small vectors ($10^1 \leq N \leq 10^5$) before the data-transfer cost becomes visible at larger sizes ($N \geq 10^7$). For small vectors, the nearly fixed launch, allocation, and transfer overhead dominates useful computation. At large N , the curves rise because ReLU reads and writes every element and is primarily constrained by memory movement. At $N = 10^8$, Array took 0.2498 seconds in total, whereas Buffer took 0.5525 seconds. Thus, Array was faster by 54.8% relative to the Buffer time.

3.2 PyCUDA

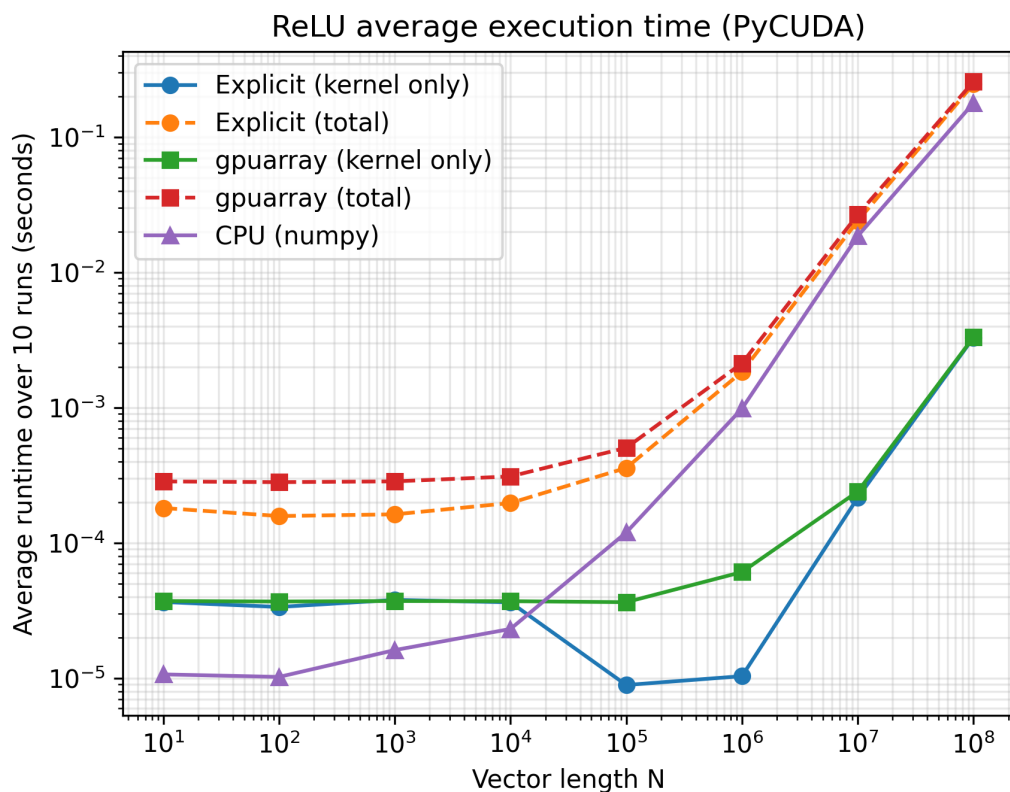


Figure 5: Average PyCUDA ReLU execution time over 10 runs.

Figure 5 shows a nearly constant cost for small vectors, followed by approximately size-proportional growth once vector size grows larger ($N \geq 10^6$). Both explicit and `gpuarray` kernel-only times are close at large N because they launch the same kernel with the same geometry. Their total times differ because their allocation and transfer wrappers have different overheads. At $N = 10^8$, Explicit took 0.2475 seconds end-to-end and `gpuarray` took 0.2580 seconds. Explicit was therefore faster by 4.07% relative to the `gpuarray` time.

4 Theory Questions

1. Synchronization

- "Asynchronous" means the host device and kernels don't work in the same time line. An asynchronous kernel launch places work into a device stream and returns control to the host before that work finishes. The host can perform independent CPU work, prepare later operations, or enqueue more GPU work instead of waiting until the kernel completes.
- No, it's not required. The device-to-host copy is enqueued after the kernel in the same stream, so stream ordering prevents the copy from reading the output early; the blocking copy also does not return to the host until the copied data is available.

- (c) Yes, it's required. The devices returns controls to the host once the kernels are enqueued, which means host will get a time without kernels executed if host doesn't wait for devices to synchronize.

2. Kernel-only versus end-to-end time

For PyOpenCL Array, the measured ratios were

$$R_{10}^{\text{OpenCL}} = \frac{6.867 \times 10^{-4}}{9.830 \times 10^{-6}} = 69.86, \quad R_{10^8}^{\text{OpenCL}} = \frac{2.498 \times 10^{-1}}{3.234 \times 10^{-3}} = 77.24.$$

For PyCUDA Explicit, they were

$$R_{10}^{\text{CUDA}} = \frac{1.811 \times 10^{-4}}{3.656 \times 10^{-5}} = 4.95, \quad R_{10^8}^{\text{CUDA}} = \frac{2.475 \times 10^{-1}}{3.300 \times 10^{-3}} = 75.00.$$

The gap between the curves consists of device allocation, host-to-device transfer, device-to-host transfer, and host/API overhead. At small N , fixed overheads dominate the extremely short kernel. As N increases, both kernel work and byte transfers scale with N . For this low-arithmetic-intensity kernel, transfers remain much more time-consuming than the kernel itself, which explains the large ratios at $N = 10^8$. The particularly large change in the CUDA ratio also reflects the comparatively high measured small- N kernel time, where the cost of event and launch effects is significant compared to useful work.

3. Array/gpuarray versus Buffer/mem_alloc

At $N = 10^8$, PyOpenCL Array was faster end-to-end than Buffer by 0.3027 seconds (54.8% relative to Buffer), while PyCUDA Explicit was faster than `gpuarray` by 0.0105 seconds (4.07% relative to `gpuarray`). The Array and `gpuarray` interfaces wrap device allocation, metadata, and data transfers in higher-level objects. Buffer and `mem_alloc` expose those operations more explicitly. Within either API, the paired implementations execute the same element-wise kernel, so their kernel-only times at $N = 10^8$ are nearly identical: 3.234 versus 3.245 ms for PyOpenCL and 3.300 versus 3.323 ms for PyCUDA. End-to-end time also includes wrapper, allocation, and copy behavior and therefore can differ.

4. GPU versus CPU crossover

Using end-to-end time, neither API beat NumPy anywhere in the tested range $10 \leq N \leq 10^8$. Even at $N = 10^8$, the fastest GPU totals were 0.2498 seconds for PyOpenCL Array and 0.2475 seconds for PyCUDA Explicit, compared with approximately 0.1794 seconds for NumPy. Using kernel-only time, both PyOpenCL implementations first beat NumPy at $N = 10$, while both PyCUDA implementations first did so at $N = 10^5$. Kernel-only time comparison favors the GPU because it excludes setup and PCIe transfers. At small N , there is too little parallel work and the GPU needs to transfer and receive data multiple times, allocate memory for elements and launch kernels, so NumPy can finish faster than the end-to-end GPU.

5. Scaling limits

For the first case, the kernel uses a finite-width index and the launch geometry has device-specific grid limits. If N exceeds the range that the index expression or one launch can cover, indices can

overflow or some elements cannot be addressed. The kernel should use a sufficiently wide index type and a grid-stride loop, or the host should split the vector into multiple launches with an offset.

For the second case, even a valid index space does not guarantee that the input and output fit in device memory. If their combined allocation exceeds available memory, allocation fails. The host should process the vector in chunks that fit, transfer each chunk, launch ReLU for that chunk, and copy it back. Pinned host buffers, multiple streams/queues, and double buffering can overlap transfers with computation. Chunk boundaries do not change the result because ReLU is element-wise.